

PROJECT DOCUMENTATION

AI FITNESS PLAN GENERATOR USING GEMINI

1.INTRODUCTION

Project Title: FITBUDDY

TeamID:

NM ID:

Name: ASVIKA N- B806FC735219660B0144E4BFD875C9A9

Name: DHEVADARSHNI T- 7041C448C932429A60047E417237D891

Name: NETHRA S- 1AE461D75726509BC0A460DF5B7247FA

Name: RITHIKA- 55487D472EBF53FB22B4D0528CE42575

Name: YAMINI J- 75E842345527FB99BB844770C0039197

TeamLeader:

Name : ASVIKA N & Mail ID: asvikamoorthy@gmail.com

TeamMembers:

Name : DHEVADARSHNI T & Mail ID: dhevadarshni1005@gmail.com

Name : RITHIKA B & Mail ID: rithikavasi2011@gmail.com

Name : NETHRA S & Mail ID: nethraoff2006@gmail.com

Name : YAMINI J & Mail ID: j.yamini1022@gmail.com



2.PROJECT OVERVIEW

● Purpose:

FitBuddy – AI Fitness Plan Generator Using Gemini is a web-based Generative AI application designed to create **personalized 7-day fitness plans** based on a user's age, weight, fitness goal, and preferred workout intensity. It uses **Google Gemini AI** to generate workout recommendations and nutrition/recovery guidance.

Main Purpose

1. Personalized Fitness Planning

Generate workout plans according to the user's personal information, fitness goal, and preferred intensity.

2. AI-Powered Workout Generation

Use Gemini AI to create a structured 7-day workout schedule with exercises, repetitions, duration, rest periods, warm-ups, and cooldowns.

3. Nutrition and Recovery Guidance

Provide AI-generated general nutrition and recovery tips that complement the user's fitness goal.

4. **Feedback-Based Plan Improvement**

Allow users to provide feedback about their workout plan and generate an updated plan according to their requirements.

5. **Easy Web-Based Access**

Provide a simple interface where users can enter their details and receive their fitness plan without needing specialized fitness-planning knowledge.

6. **Data Storage and Management**

Store user information, generated plans, and feedback using a database so previous plans can be managed.

● **Features:-**

- Personalized workout plans
- Gemini AI integration
- 7-day workout generation
- Nutrition & recovery tips
- Feedback-based plan updates
- User data storage
- Admin dashboard
- Responsive web interface
- REST API
- Input validation and safety guidance

3.ARCHITECTURE

● **Frontend:** The frontend is designed using HTML and CSS with Jinja2 templates. The main pages include:

- Home page.
- Workout result page.
- Feedback section.
- Administrator dashboard.

The home page collects user information. The result page displays the generated seven-day workout plan and nutrition/recovery information.

● **Backend:** FastAPI is used as the backend framework. It manages:

- HTTP requests.
- Form processing.
- Data validation.
- Gemini communication.
- Database operations.
- API endpoints.
- Administrative authentication.

● **Database:** SQLite is used as the database because it is lightweight and suitable for the project.

The main database entities are:

USER

```
User
-----
id
user_id
username
age
weight
goal
intensity
created_at
```

PLAN

```
Plan
-----
id
user_id
original_plan
updated_plan
nutrition_tip
feedback
created_at
updated_at
```

● AI Design

The AI component is divided into separate services.

The main Gemini planner generates the seven-day workout plan.

A separate Gemini service generates nutrition and recovery guidance.

Another AI service processes user feedback and produces an updated workout plan.

This modular approach makes the application easier to maintain and extend.

4.SET UP INSTRUCTIONS

● ? Install Python 3.11

● ? Open the FitBuddy project in VS Code

● ? Create a virtual environment

```
python -m venv .venv
```

```
.\.venv\Scripts\Activate.ps1
```

● Install dependencies

```
pip install -r requirements.txt
```

● ? Create .env and add your Gemini API key.

● ? Run the application

```
uvicorn app.main:app --reload
```

? Open in browser: <http://127.0.0.1:8000>

? API documentation: <http://127.0.0.1:8000/docs>

? Run tests

● Installation Steps:

1. Install Python 3.11
2. Open the project folder in VS Code
3. Open the VS Code Terminal
4. Create a virtual environment

```
python -m venv .venv
```

5. Activate the virtual environment

```
.\venv\Scripts\Activate.ps1
```

6. Install required packages

```
pip install -r requirements.txt
```

7. Create the .env file and configure the Gemini API key.

8. Start the application

```
uvicorn app.main:app --reload
```

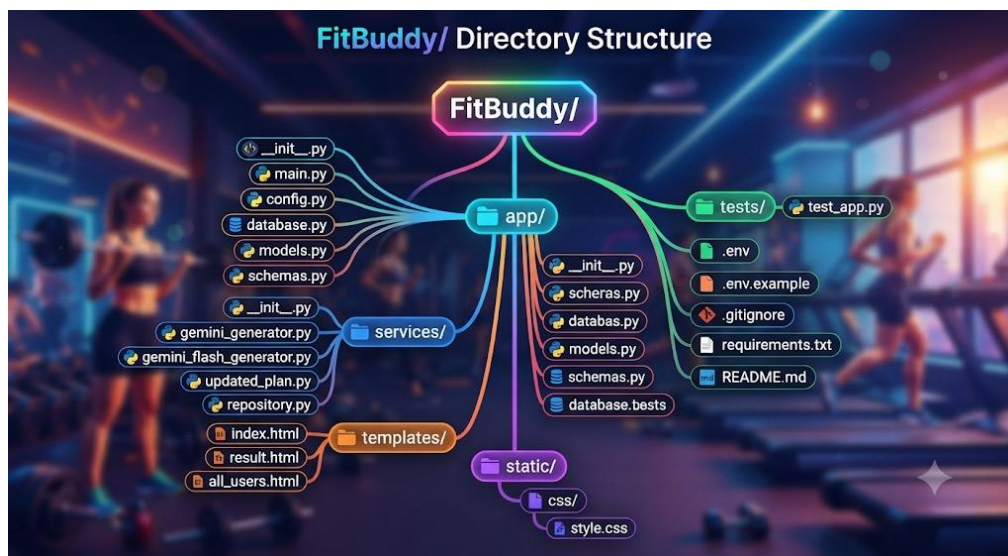
9. Open the application

```
http://127.0.0.1:8000
```

10. Run tests

```
pytest -
```

5.PROJECT STRUCTURE



6.RUNNING APPLICATIONS

1. Open the FitBuddy folder in VS Code.
2. Open the terminal and activate the virtual environment:

```
.\venv\Scripts\Activate.ps1
```

3. Start the FastAPI server:

```
uvicorn app.main:app --reload
```

4. Open the application in your browser:

<http://127.0.0.1:8000>

5. API documentation:

<http://127.0.0.1:8000/docs>

6. Admin dashboard:

<http://127.0.0.1:8000/view-all-users>

7. To stop the application, press:

Ctrl + C

7.API DOCUMENTATION

FitBuddy provides REST API endpoints for generating workout plans, submitting feedback, and checking application status.

Method	Endpoint	Purpose
GET	/api/health	Check API/server status
POST	/api/generate-workout	Generate a personalized 7-day workout plan
POST	/api/submit-feedback	Update the workoutplan using user feedback

Swagger API Documentation

After starting the application, open:

<http://127.0.0.1:8000/docs>

This opens Swagger UI, where you can view endpoints, request parameters, schemas, and test the APIs directly.

Alternative API Documentation

FastAPI also provides ReDoc:

<http://127.0.0.1:8000/redoc>

SOURCE CODE

Code for main.py

```
from database import init_db
from contextlib import asynccontextmanager
from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles
from config import get_settings
from database import init_db
@asynccontextmanager
async def lifespan(app: FastAPI):
    # Initializes our SQLite database layout automatically when the application starts
    init_db()
    yield
    settings = get_settings()
    app = FastAPI(
        title=settings.app_name,
        version="1.0.0",
        description="FitBuddy-AI-powered fitness plan generator.",
        lifespan=lifespan,
    )
    # Safely includes the router engine after initialization block to bypass loops
    from routes import router
    app.include_router(router)
```

Code for .env

```
GEMINI_API_KEY=AQ.Ab8RN6LvOsT1tDbIfo3NPGScV-xtjSCfyqmAfQoNkHV5yW9sJg
DATABASE_URL=sqlite:///./fitbuddy.db GEMINI_WORKOUT_MODEL=gemini-2.5-pro
GEMINI_TIP_MODEL=gemini-3.8-flash
ADMIN_TOKEN=admin
APP_NAME=FitBuddy
```

INDEX HTML CODE

```
<!DOCTYPEhtml>

<htmllang=
"en">

<head>

<metacharset=
"UTF-8">

<title>FitBuddyAI-Registration</title>

<style>

body{font-family:'SegoeUI'
,sans-serif;background-color:#edf2f7;margin:0;padding:40px;
display:flex; justify-content:center;align-items:center;min-height:90vh;}

.container{background:white;padding:35px;border-radius:16px;box-shadow:0
10px25px
rgba(0,0,0,0.05);max-
width:500px;width:100%;border:1pxsolid#d2e4f6;position:relative;}

.ai-mascot{position:absolute;top: -
45px;right:20px;width:85px;height:85px;background:white;
border-radius:50%;box-shadow:04px10pxrgba(0,0,0,0.1);display:flex; justify-
content:center;
align-items: center;font-size:45px;border:3pxsolid#3498db;}

h1{color:#2c3e50;text-align: center;margin-bottom:5px;}

.subtitle{ color:#7f8c8d;text-align: center;margin-bottom:30px;font-size:14px;}

.form-row{display:flex;gap:15px;}

.form-group {margin-bottom:20px;flex:1;}

label{display:block;margin-bottom:8px;font-weight:600;color:#34495e;font-
size:14px;}

input, select{width:100%;padding:12px;border:1pxsolid#cbd5e0;border-
radius:8px;box-sizing:
border-box;font-size:14px;background-color:#f8f9fa;}
```

```
button{width:100%;background-
color:#3498db;color:white;border:none;padding:14px;border-
radius:8px;font-size:16px;font-weight:600;cursor:pointer;margin-top:10px;}
button:hover{background-color:#2980b9;}
.admin-link{display:block;text-align: center;margin-top:15px;color:#7f8c8d;font-
size:13px;text-
decoration: none;}
</style>
</head>
<body>
<div class=
"container">
<div class=
"ai-mascot"><img alt="AI Mascot" data-bbox="242 454 268 471"/></div>
<h1>FitBuddyAI</h1>
<pclass=
"subtitle">Initialize your tailored workout adventure profile</p>
<form action=
"/generate-plan" method=
"POST">
<div class=
"form-group">
<label for=
"name">Full Name</label>
<input type=
"text" id=
"name" name=
"name" placeholder=
```

```
"e.g.AlexRivera"required> </div>
```

```
<divclass=
```

```
"form-row">
```

```
<divclass=
```

```
"form-group">
```

```
<labelfor=
```

```
"age">Age</label>
```

```
<inputtype=
```

```
"number"id=
```

```
"age"name=
```

```
"age"placeholder=
```

```
"25"required>
```

```
</div>
```

```
<divclass=
```

```
"form-group">
```

```
<labelfor=
```

```
"gender">Gender</label>
```

```
<selectid=
```

```
"gender"name=
```

```
"gender">
```

```
<optionvalue=
```

```
"Male">Male</option>
```

```
<optionvalue=
```

```
"Female">Female</option>
```

```
<optionvalue=
```

```
"Non-Binary">Non-Binary</option>
```

```
</select>
```

```
</div>
```

</div>

<divclass=

"form-row">

<divclass=

"form-group">

<labelfor=

"weight">Weight(kg)</label>

<inputtype=

"number" id=

"weight" name=

"weight" placeholder=

"65" required>

</div>

<divclass=

"form-group">

<labelfor=

"height">Height(cm)</label>

<inputtype=

"number" id=

"height" name=

"height" placeholder=

"170" required>

</div>

</div>

<divclass=

"form-group">

<labelfor=

"days">WorkoutDaysPerWeek</label>

```
<selectid=
"days"name=
"days">
<optionvalue=
"3">3Days(FoundationalSplit)</option>
<optionvalue=
"4">4Days(OptimizedTargetSplit)</option>
<optionvalue=
"5">5Days(AdvancedIntensitySplit)</option>
</select>
</div>
<divclass=
"form-group">
<labelfor=
"goal">Objective</label>
<selectid=
"goal"name=
"goal">
<optionvalue=
"WeightLoss">Weight Loss 📉</option>
<optionvalue=
"MuscleGain">MuscleGain 💪</option>
<optionvalue=
"Endurance">EnduranceTraining 🏃 ♂ </option>
<optionvalue=
"GeneralHealth">GeneralFitness 🌟</option>
</select>
</div>
```

```
<div class=  
"form-group">  
  <label for=  
"level">Experience Level</label>  
  <select id=  
"level" name=  
"level">  
    <option value=  
"Beginner">Beginner (Foundational)</option>  
    <option value=  
"Intermediate">Intermediate (Rising Star)</option>  
    <option value=  
"Advanced">Advanced (Elite Level)</option> </select>  
  </div>  
  <button type=  
"submit">Generate Custom Blueprint</button>  
</form>  
  <a href=  
"/users" class=  
"admin-link">🛡️ View Database Admin Registry</a>  
</div>  
</body>  
</html>
```

Code for routes.py

Code for routes.py

```
import os

import random

from fastapi import APIRouter, Form, Request, HTTPException

from fastapi.responses import HTMLResponse

from fastapi.templating import Jinja2Templates

from database import get_db_connection

router = APIRouter()

# 📍 Dynamically locate the templates directory inside app/templates
CURRENT_DIR = os.path.dirname(os.path.abspath(__file__))

TEMPLATE_DIR = os.path.join(CURRENT_DIR,

                             "templates")

templates = Jinja2Templates(directory=TEMPLATE_DIR)

# 1. Frontpage router link

@router.get("/")

, response_class=HTMLResponse)

async def read_root(request: Request):

    return templates.TemplateResponse(request=request, name=

    "index.html")

# 2. Main workflow registration generator endpoint

@router.post("/generate-plan"

, response_class=HTMLResponse)

async def generate_plan(

    request: Request,

    name: str = Form(...),

    age: int = Form(...),

    gender: str = Form(...),
```

```

weight:float =Form(...),
height:float=Form(...),
days: int =Form(...),
goal: str=Form(...),
level: str=Form(...)
):
generated_schedule=f"
===FITBUDDYWORKOUTBLUEPRINTFOR{name.upper()}===\n"
generated_schedule+=f"TargetParameters:{days}Daysperweeksplitfocusingon{goal}.\n\n"
generated_schedule+=
"Day1:UpperBodyPowerSplitRoutine\n-BenchPress(3setsx10 reps)\n-
DumbbellRows(3setsx12reps)\n\n"
generated_schedule+=
"Day2: LowerBodyFoundationalSplitRoutine\n-Squats(3setsx12reps)\n-
RomanianDeadlifts(3 setsx10 reps)\n\n"
ifdays>=3:
generated_schedule+=
"Day3:CoreIsolation&HIITConditioning\n-PlankHold Sequences(3sets
x60 seconds)\n\n"
ifdays>=4:
generated_schedule+=
"Day4:PosteriorChainVariableFocus\n-InclineDumbbellPress(3setsx
10reps)\n\n"
ifdays>=5:
generated_schedule+=
"Day5:AgilityCapacities&CoreBlast\n-KettlebellSwings(4setsx12
reps)\n"
#Savecleanlyintothedatabaserecords

```

```

conn=get_db_connection()
cursor=conn.cursor()
cursor.execute("""
INSERTINTOusers(name,age,gender,weight,height,days,goal,level,original_plan)
VALUES(?, ?, ?, ?, ?,?,?,,?)
""")

,(name, age,gender,weight,height,days,goal, level,generated_schedule))
conn.commit()
new_id=cursor.lastrowid
#ForceconvertdatabaseRowstructureintostandardpythondictionary
row=cursor.execute("SELECT* FROMusersWHEREid=?"
,(new_id,)).fetchone()
user_dict=dict(row)
conn.close()
returntemplates.TemplateResponse(
request=request,
name=
"dashboard.html"
,
context={
"user": user_dict,
"alert_msg":"Profile configurationregisteredand loggedsuccessfully
intodatabase!"
}
)
#3.Dynamiclayout dashboardfetchtoolURLendpoint
@router.get("/user/{user_id}"
,response_class=HTMLResponse)

```

```

asyncdef view_user_dashboard(request:Request,user_id:int):
    conn=get_db_connection()
    row=conn.execute("SELECT*FROMUsersWHEREid ="
    ,(user_id,)).fetchone()
    conn.close()
    if not row: raise HTTPException(status_code=404,detail=
    "Profile registry ID not found.")
    return templates.TemplateResponse(
    request=request,
    name=
    "dashboard.html"
    ,
    context={"user":dict(row)}
    )

```

#4.InteractiveFeedbackhandlerpipeline route configuration

```

@router.post("/submit-feedback"
, response_class=HTMLResponse)
asyncdef submit_feedback(
    request:Request,
    user_id:int=Form(...),
    feedback_text: str=Form(...)
):
    conn=get_db_connection()
    cursor=conn.cursor()
    row=cursor.execute("SELECT* FROMUsersWHEREid ="
    ,(user_id,)).fetchone()
    if not row:
    conn.close()

```

```

raiseHTTPException(status_code=404,detail=
"Userverificationkey failed.")
user_data=dict(row)
base_plan=user_data["original_plan"]
adapted_plan=f"
===ADAPTEDPROFILEBLUEPRINT(REVISEDVIAUSERFEEDBACK)
===\nFeedback: \"{feedback_text}\"\\n\\n"+base_plan
cursor.execute("UPDATEUsersSETupdated_plan=?,feedback_text=?WHEREid=?"
,
(adapted_plan,feedback_text,user_id))
conn.commit()
updated_row =cursor.execute("SELECT* FROMUsersWHERE id=?"
,(user_id,)).fetchone()
conn.close()
returntemplates.TemplateResponse(
request=request,
name=
"dashboard.html"
,
context={
"user":dict(updated_row),
"alert_msg":"Yourplanhasbeenupdated basedonyourfeedback!"
}
)
#5.GlobalViewAllUsersadministrativetrackingrouteendpoint
@router.get("/users"
,response_class=HTMLResponse)
asyncdef view_all_users(request:Request):

```

```

conn=get_db_connection() rows=conn.execute("SELECT*
FROMUsersORDERBYidDESC").fetchall()

conn.close()

#Packrowscleanlyinto list ofdictionarynodesto feedJinja2natively
all_users=[dict(r)forrinrows]

returntemplates.TemplateResponse(
request=request,
name=
"admin.html"
,
context={"users": all_users}
)

```

Code for database.py

Code for database.py:

```

import sqlite3
import os

CURRENT_DIR=os.path.dirname(os.path.abspath(__file__))
BASE_DIR=os.path.dirname(CURRENT_DIR)
DB_PATH =os.path.join(BASE_DIR,
"fitbuddy.db")

definit_db():

conn=sqlite3.connect(DB_PATH)

cursor=conn.cursor()

cursor.execute("""

CREATETABLEIFNOTEXISTSusers(

id INTEGERPRIMARYKEYAUTOINCREMENT,

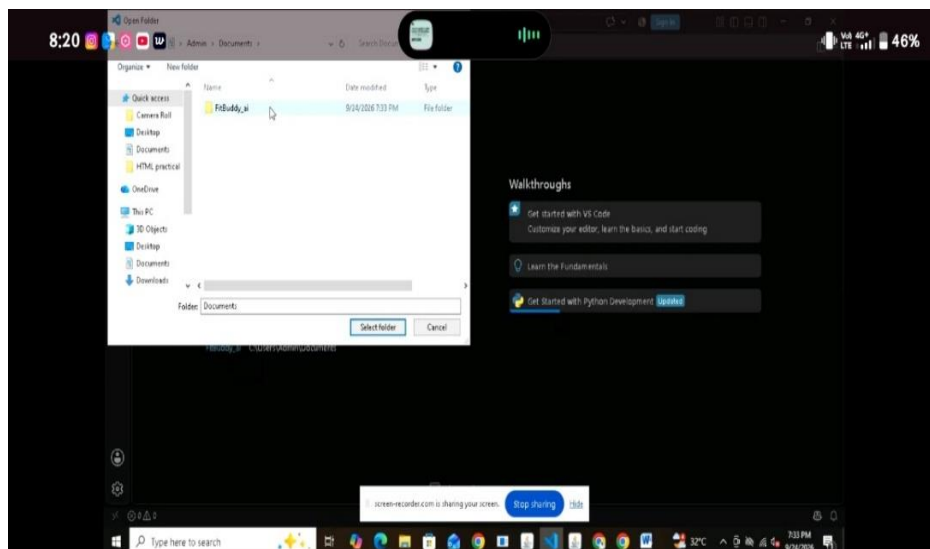
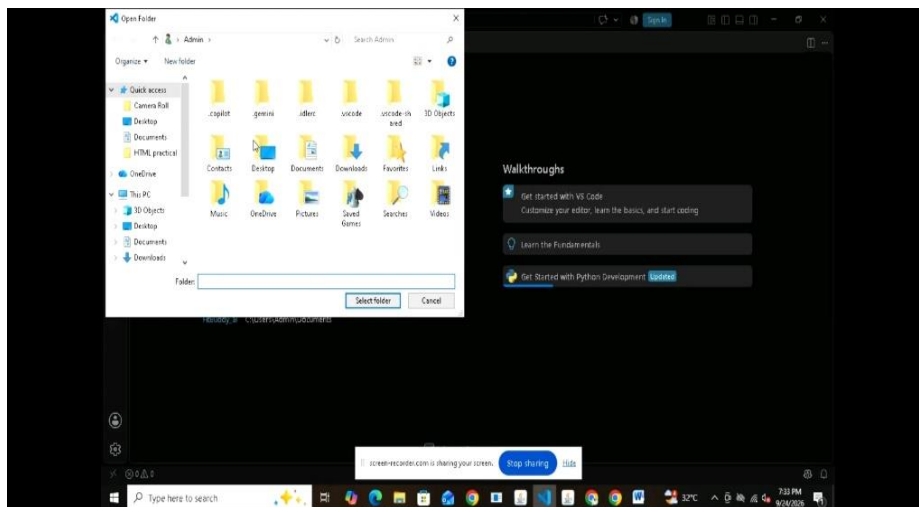
nameTEXTNOTNULL,

age INTEGERNOTNULL,

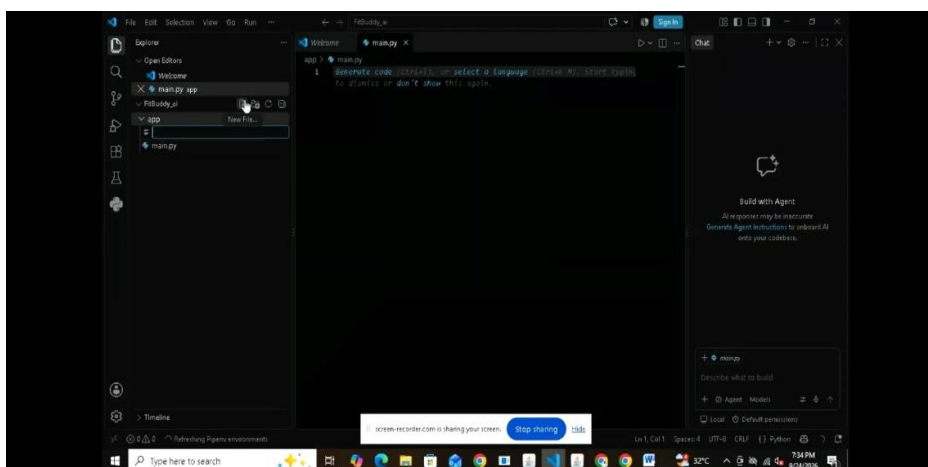
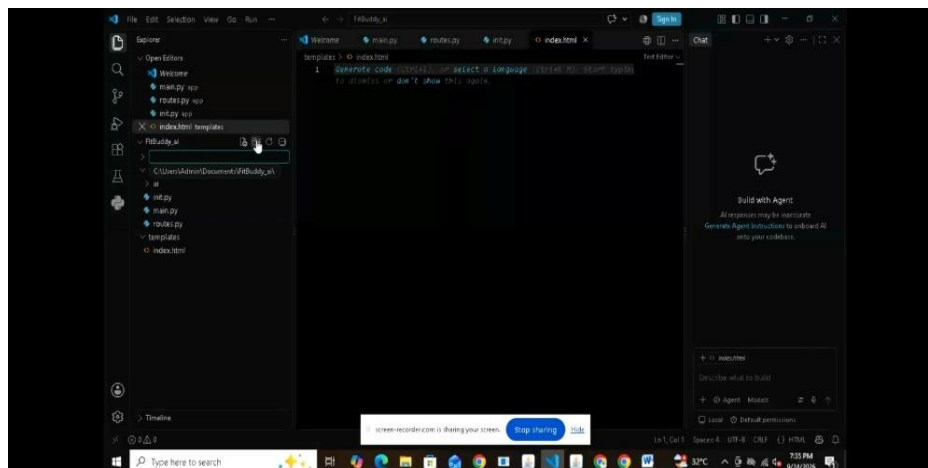
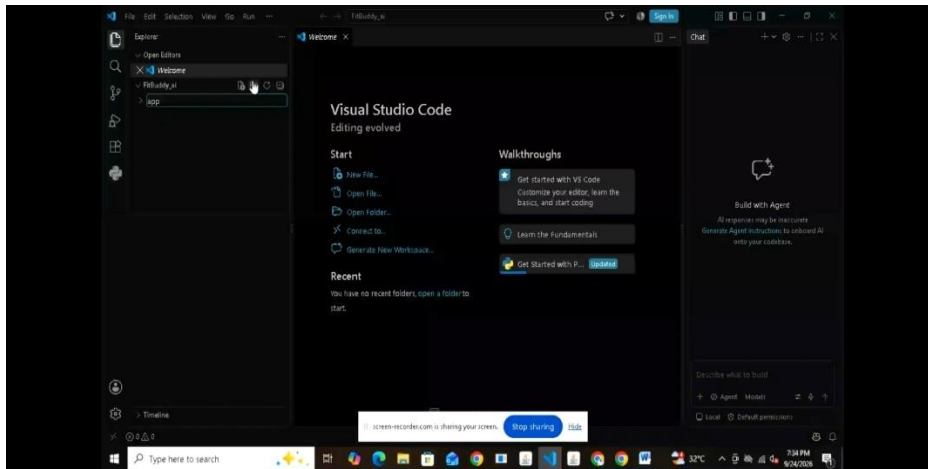
```

```
genderTEXTNOTNULL,  
weightREALNOTNULL,  
heightREALNOTNULL,  
daysINTEGERNOTNULL,  
goalTEXTNOTNULL,  
levelTEXTNOTNULL,s  
original_planTEXTNOTNULL,  
updated_planTEXT,  
feedback_textTEXT  
)  
""")  
conn.commit()  
conn.close()  
defget_db_connection():  
conn=sqlite3.connect(DB_PATH)  
conn.row_factory=sqlite3.Row  
returnconn
```

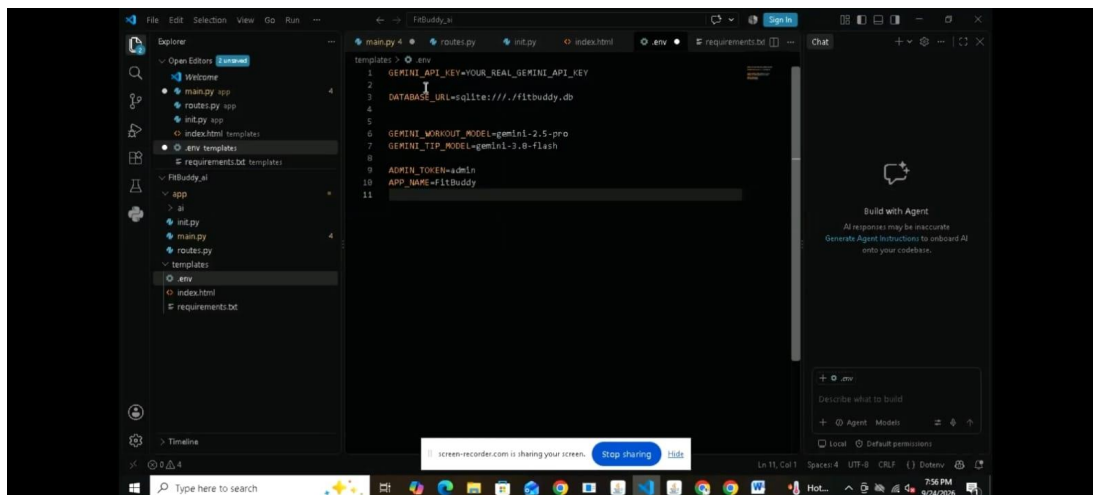
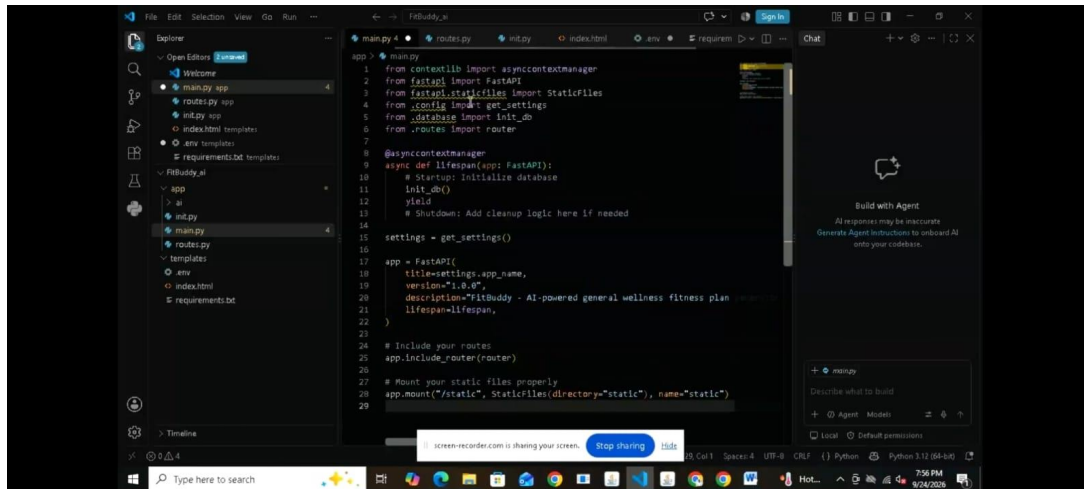
1.FOLDER CREATION



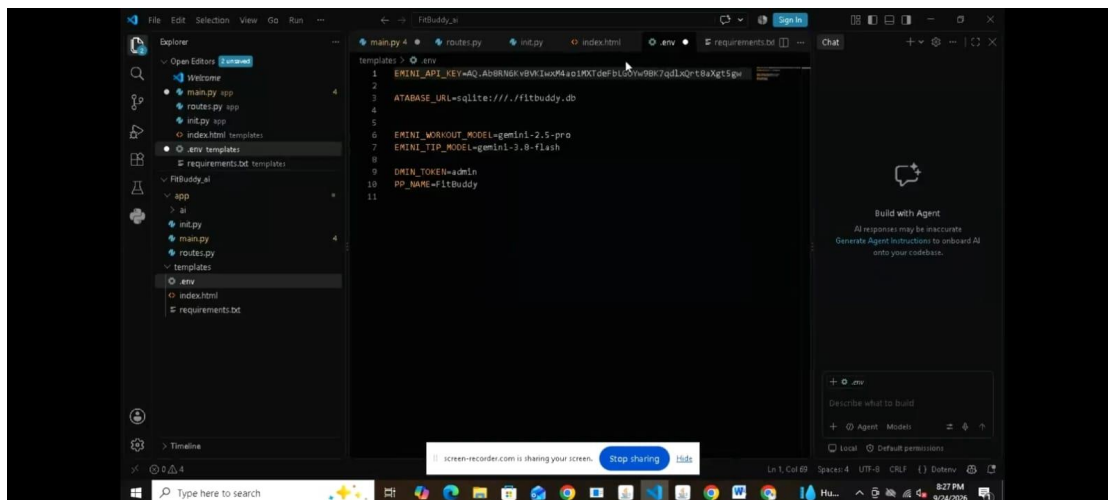
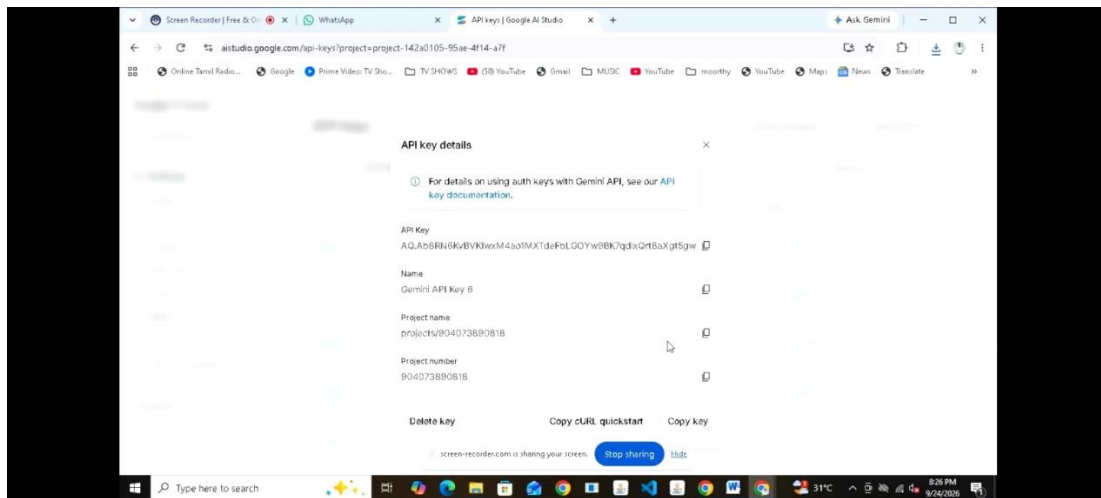
2.VISUAL STUDIO



3.CODING

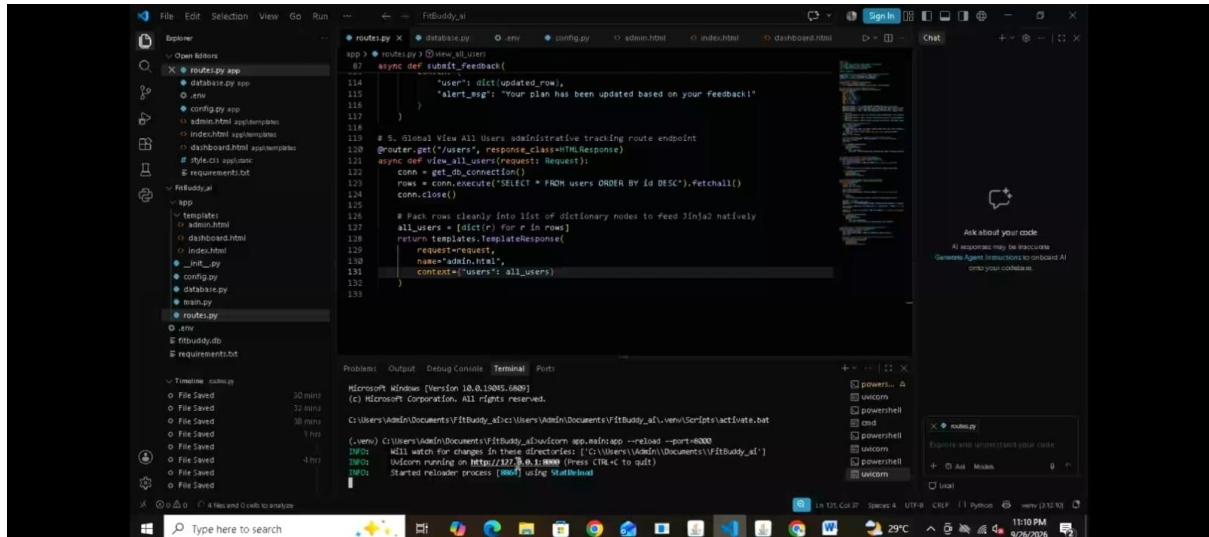


4.APP KEY



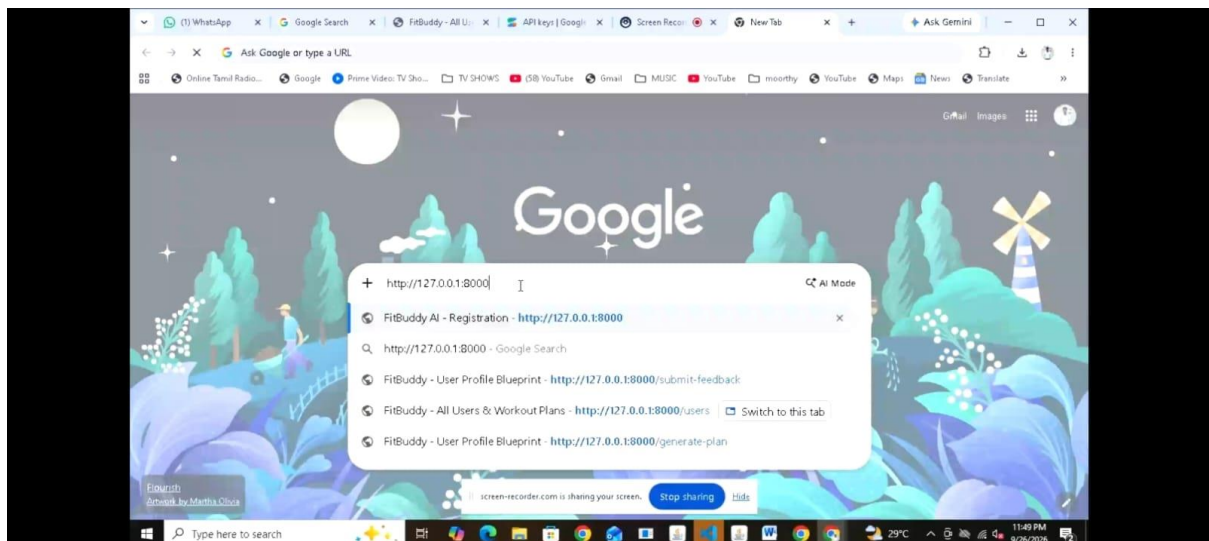
5.RUNNING INTRUCTIONS

uvicorn app.main:app --reload --port=8000



6.BROWSING LINK

<http://127.0.0.1:8000/users>



7.WEB PAGE

The screenshot shows a web browser window with the FitBuddy AI registration form. The browser's address bar displays the IP address 127.0.0.1:8000. The form is titled "FitBuddy AI" and includes the subtitle "Initialize your tailored workout adventure profile". The form fields are as follows:

- Full Name:** e.g. Alex Rivera
- Age:** 25
- Gender:** Male
- Weight (kg):** 65
- Height (cm):** 170
- Workout Days Per Week:** 3 Days (Foundational Split)
- Objective:** Weight Loss
- Experience Level:** Beginner (Foundational)

The Windows taskbar at the bottom shows the search bar, task view button, and several application icons. The system tray on the right indicates a temperature of 29°C and the date/time as 10:32 PM on 9/26/2025.

This screenshot shows the same FitBuddy AI registration form, but with a different user profile entered. The fields are filled with the following information:

- Full Name:** Asvika
- Age:** 18
- Gender:** Female
- Weight (kg):** 50
- Height (cm):** 160
- Workout Days Per Week:** 4 Days (Optimized Target Split)
- Objective:** General Fitness
- Experience Level:** Beginner (Foundational)

At the bottom of the form, there is a prominent blue button labeled "Generate Custom Blueprint". The browser window and Windows taskbar are identical to the previous screenshot.

8.USER INFORMATION

A screenshot of a web browser displaying the FitBuddy AI website. The browser's address bar shows the URL "127.0.0.1:8000". The website has a light blue background. At the top, there is a navigation bar with a search icon, a star icon, and a list of icons for various services: Online Tamil Radio, Google, Prime Video, TV SHOWS, YouTube, Gmail, MUSIC, YouTube, moorthy, YouTube, Maps, News, and Translate. Below the navigation bar, there is a circular profile picture of a person with a red and white headband. The main content area features the "FitBuddy AI" logo and the tagline "Initialize your tailored workout adventure profile". Below this, there is a registration form with the following fields: "Full Name" (with the value "Arvika"), "Age" (with the value "18"), "Gender" (with a dropdown menu showing "Female"), "Weight (kg)" (with the value "45"), "Height (cm)" (with the value "150"), "Workout Days Per Week" (with a dropdown menu showing "3 Days (Foundational Split)"), "Objective" (with a dropdown menu showing "General Fitness"), and "Experience Level" (with a dropdown menu showing "Intermediate (Rising Star)"). At the bottom of the form, there is a blue button labeled "Generate Custom Blueprint" and a link that says "View Database Admin in Registry".

FITNESS PLAN OUTPUT

The screenshot shows a web browser window with the address bar displaying '127.0.0.1:8000/generate-plan'. The browser's tab bar includes several open tabs, and the address bar shows various search engines and social media links. The main content area is titled 'User Profile Blueprint Dashboard' and features a green success message: 'Profile configuration registered and logged successfully into database log!'. Below this, the user's profile information is displayed in two columns. The left column includes 'User Registration ID: 2', 'Age Metric: 18 yrs', 'Weight Metrics: 45.0 kg', and 'Core Objective: General Health'. The right column includes 'Profile Name: Asvika', 'Gender: Female', 'Height Index: 150.0 cm', and 'Experience Level: Intermediate'. A 'Nutrition Tip' section is also visible, advising on high-value dietary protein loops. At the bottom, a 'Current' status bar indicates that the screen recorder is sharing the screen, with a 'Stop sharing' button and a 'Hide' link.

Registration ID	Profile Name
2	Asvika

Age Metric	Gender
18 yrs	Female

Weight Metrics	Height Index
45.0 kg	150.0 cm

Core Objective	Experience Level
General Health	Intermediate

Nutrition Tip
 Prioritize high-value dietary protein loops: Aim for a clean, regular source of lean amino matrices (like grilled poultry breast, fresh ocean catch, clean legumes, or Greek yogurt blends) structural alignment inside every targeted meal log block. Balanced nutrition repairs broken muscle tissue fibers, maintains macro saturation profiles, and boosts operational calorie burn velocity across both fat reduction and athletic bulking regimens!

Current | screen-recorder.com is sharing your screen. | [Stop sharing](#) | [Hide](#)

9..NUTRITION GUIDE

FitBuddy - All Users & Workout Plans

USER ID	NAME	AGE	METRICS LOG	GOAL SUMMARY	ORIGINAL WORKOUT PLAN	UPDATED ADAPTIVE PLAN
2	Arvika	18	Gen Profile Wt: 45.8 kg Ht: 158.8 cm	General Health Intermediate 3 Days/Week	=== FITBUDDY WORKOUT BLUEPRINT FOR ARVIKA === Target Parameters: 3 Days per week split Focusing on General Health. Day 1: Upper Body Power Split Routine - Bench Press (3 sets x 10 reps) - Dumbbell Rows (3 sets x 12 reps) Day 2: Lower Body Foundational Split Routine - Squats (3 sets x 12 reps) - Romanian Deadlifts (3 sets x 10 reps) Day 3: Core Isolation & HIIT Conditioning - Plank Hold Sequences (3 sets x 60 seconds)	=== ADAPTED PROFILE BLUEPRINT (REVISED VIA USER FEEDBACK) === Feedback: "I fell down and broke my leg, So I cant practice the exercise, So what should I do?" === FITBUDDY WORKOUT BLUEPRINT FOR ARVIKA === Target Parameters: 3 Days per week split Focusing on General Health. Day 1: Upper Body Power Split Routine - Bench Press (3 sets x 10 reps) - Dumbbell Rows (3 sets x 12 reps) Day 2: Lower Body Foundational Split Routine - Squats (3 sets x 12 reps) - Romanian Deadlifts (3 sets x 10 reps) Day 3: Core Isolation & HIIT Conditioning - Plank Hold Sequences (3 sets x 60 seconds)

screen-recorder.com is sharing your screen. [Stop sharing](#) [Hide](#)

4: "I fell down and broke my leg. So I cant practice the exercise, So what should I do?"

FitBuddy - All Users & Workout Plans

USER ID	NAME	AGE	METRICS LOG	GOAL SUMMARY	ORIGINAL WORKOUT PLAN	UPDATED ADAPTIVE PLAN
1	Arvika	18	Gen Profile Wt: 45.8 kg Ht: 158.8 cm	General Health Intermediate 3 Days/Week	=== FITBUDDY WORKOUT BLUEPRINT FOR ARVIKA === Target Parameters: 3 Days per week split Focusing on General Health. Day 1: Upper Body Power Split Routine - Bench Press (3 sets x 10 reps) - Dumbbell Rows (3 sets x 12 reps) Day 2: Lower Body Foundational Split Routine - Squats (3 sets x 12 reps) - Romanian Deadlifts (3 sets x 10 reps) Day 3: Core Isolation & HIIT Conditioning - Plank Hold Sequences (3 sets x 60 seconds)	=== ADAPTED PROFILE BLUEPRINT (REVISED VIA USER FEEDBACK) === Feedback: "I fell down and broke my leg, So I cant practice the exercise, So what should I do?" === FITBUDDY WORKOUT BLUEPRINT FOR ARVIKA === Target Parameters: 3 Days per week split Focusing on General Health. Day 1: Upper Body Power Split Routine - Bench Press (3 sets x 10 reps) - Dumbbell Rows (3 sets x 12 reps) Day 2: Lower Body Foundational Split Routine - Squats (3 sets x 12 reps) - Romanian Deadlifts (3 sets x 10 reps) Day 3: Core Isolation & HIIT Conditioning - Plank Hold Sequences (3 sets x 60 seconds)
2	Arvika	18	Gen Profile Wt: 45.8 kg Ht: 158.8 cm	General Health Intermediate 3 Days/Week	=== FITBUDDY WORKOUT BLUEPRINT FOR ARVIKA === Target Parameters: 3 Days per week split Focusing on General Health. Day 1: Upper Body Power Split Routine - Bench Press (3 sets x 10 reps) - Dumbbell Rows (3 sets x 12 reps) Day 2: Lower Body Foundational Split Routine - Squats (3 sets x 12 reps) - Romanian Deadlifts (3 sets x 10 reps) Day 3: Core Isolation & HIIT Conditioning - Plank Hold Sequences (3 sets x 60 seconds)	=== ADAPTED PROFILE BLUEPRINT (REVISED VIA USER FEEDBACK) === Feedback: "I fell down and broke my leg, So I cant practice the exercise, So what should I do?" === FITBUDDY WORKOUT BLUEPRINT FOR ARVIKA === Target Parameters: 3 Days per week split Focusing on General Health. Day 1: Upper Body Power Split Routine - Bench Press (3 sets x 10 reps) - Dumbbell Rows (3 sets x 12 reps) Day 2: Lower Body Foundational Split Routine - Squats (3 sets x 12 reps) - Romanian Deadlifts (3 sets x 10 reps) Day 3: Core Isolation & HIIT Conditioning - Plank Hold Sequences (3 sets x 60 seconds)
3	Arvika	18	Gen Profile Wt: 45.8 kg Ht: 158.8 cm	General Health Intermediate 3 Days/Week	=== FITBUDDY WORKOUT BLUEPRINT FOR ARVIKA === Target Parameters: 3 Days per week split Focusing on General Health. Day 1: Upper Body Power Split Routine - Bench Press (3 sets x 10 reps) - Dumbbell Rows (3 sets x 12 reps) Day 2: Lower Body Foundational Split Routine - Squats (3 sets x 12 reps) - Romanian Deadlifts (3 sets x 10 reps) Day 3: Core Isolation & HIIT Conditioning - Plank Hold Sequences (3 sets x 60 seconds)	=== ADAPTED PROFILE BLUEPRINT (REVISED VIA USER FEEDBACK) === Feedback: "I fell down and broke my leg, So I cant practice the exercise, So what should I do?" === FITBUDDY WORKOUT BLUEPRINT FOR ARVIKA === Target Parameters: 3 Days per week split Focusing on General Health. Day 1: Upper Body Power Split Routine - Bench Press (3 sets x 10 reps) - Dumbbell Rows (3 sets x 12 reps) Day 2: Lower Body Foundational Split Routine - Squats (3 sets x 12 reps) - Romanian Deadlifts (3 sets x 10 reps) Day 3: Core Isolation & HIIT Conditioning - Plank Hold Sequences (3 sets x 60 seconds)

screen-recorder.com is sharing your screen. [Stop sharing](#) [Hide](#)

10..FEEDBACK

